

Field notes

from the branch garden

SPECIMEN 001 / MODULES II



Nine draft PRs.
One growing idea.

Still under cultivation.

A quieter treatment: botanical specimen sheet, generous margins, marginal numbers, one idea at a time. The unfinished system is something to observe closely.

DRAFT MODULES / EXPERIMENTAL FUMOLA

Names that keep their meaning.

I. A BOUNDARY

A sealed module gives an abstract type a path: **M.T**. The name says where its identity comes from, even when its representation stays hidden. Modules II is still working out the rules around that boundary.

II. A MEMORY

A Fumola livelit names a runtime and, in one form, a thunk. Reusing that name can keep an edit attached to prior state. Copying or reloading raises a different question: which history should the new object inherit?



```
# TWO LANGUAGES, ONE NOTEBOOK #  
  
let gcd : Int =  
  ^fumola_put_force(0, "`zine_gcd", "Gcd.gcd(12, 18)") in  
  
let answer : Int =  
  ^fumola_eval(0, "`zine_counter := 41; get(`zine_counter) + 1") in  
  
(gcd, answer)
```

(6, 42)

FIELD OBSERVATION 02 / Real preview, staged source. The output is (6, 42). A successful boundary crossing, not a measurement of reuse.

The shared theme is an editorial reading: a name is doing more than labeling a thing. It is keeping track of what is allowed to stay the same.

[Abstract types #2495](#) / [Fumola #2492](#) / [Runtime identity #2509](#)

NIGHT SHIFT

THE MACHINES HAVE A WORKBENCH

277

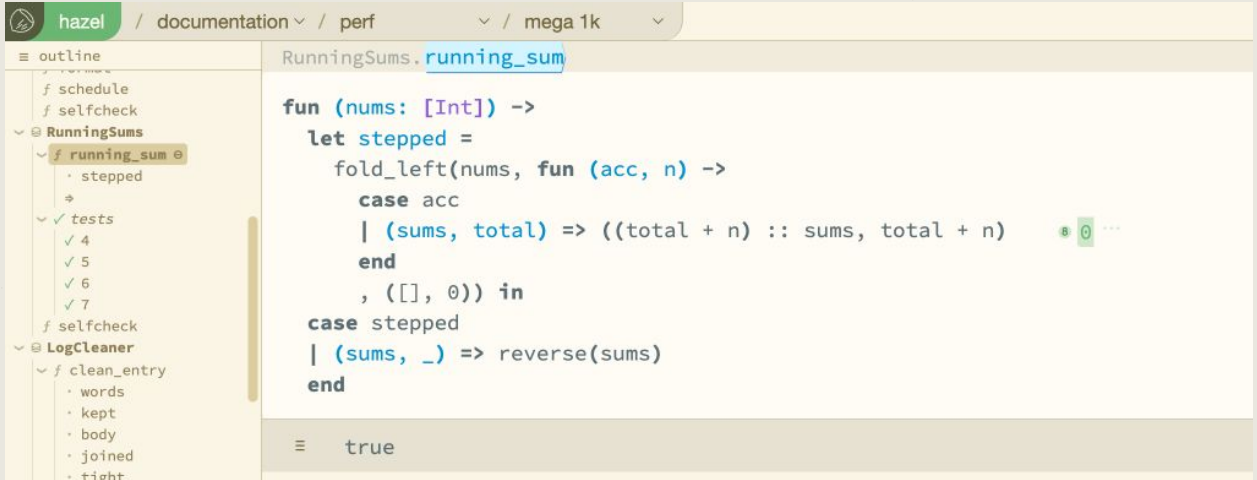
**visible Claude co-author credits
among 382 commit objects**

*No, this is not the percentage
of Hazel “written by AI.”*

It is a footprint in the metadata. Some work leaves one; some does not. Rebases, merges and cherry-picks also leave tracks. The interesting question is what people are building with the machines.

DRAFT WORKSPACE / OPEN AGENT HARDENING

Somewhere to watch the edits happen.



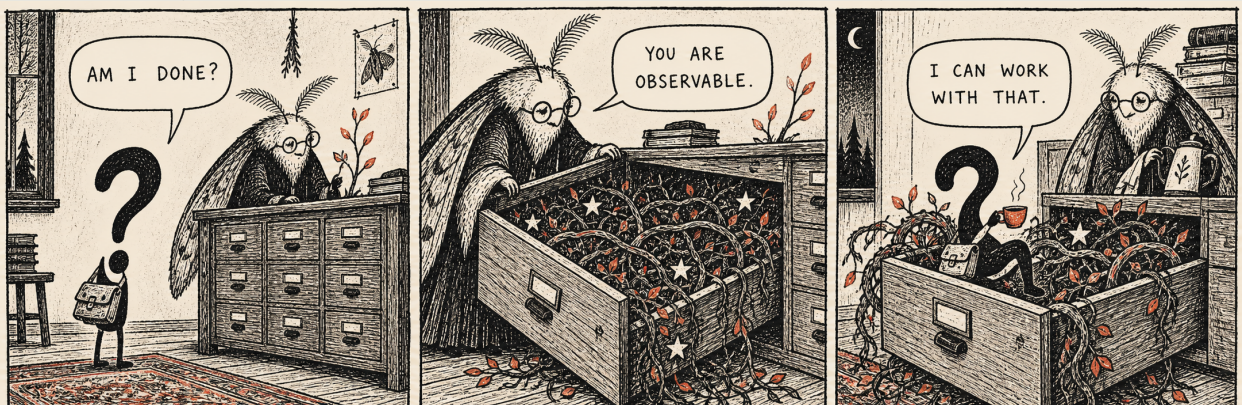
The screenshot shows the Hazel modular editor interface. The top bar displays the project path: `hazel / documentation / perf / mega 1k`. On the left, an 'outline' pane lists the file's structure, including sections like `schedule`, `selfcheck`, `RunningSums`, `tests`, `LogCleaner`, and `clean_entry`. The `RunningSums` section is expanded, showing a `running_sum` function. The main editor pane displays the code for `RunningSums.running_sum`, which is a function that takes a list of integers and returns a stepped list. The code is as follows:

```
fun (nums: [Int]) ->
  let stepped =
    fold_left(nums, fun (acc, n) ->
      case acc
      | (sums, total) => ((total + n) :: sums, total + n)
      end
      , ([], 0)) in
  case stepped
  | (sums, _) => reverse(sums)
end
```

Below the code, a status bar shows `≡ true`.

A thousand-line program, one definition on the desk. The rest of the document keeps supplying live context. That is the attraction of modular editors; Constellation tries to give the moving work a map.

FICTION DEPARTMENT



[Modular editors #2469](#) / [Constellation #2505](#) / [Hardening #2427](#)